Name : Soni Samarth

Roll no : 25MML0004

Github :

https://github.com/samarthsoni1411/25MML0004_SAMARTH-SONI_MACSE604_EDGE-INTELLIGENCE.git

Edge Intelligence assessment 2

Aim : Load and preprocess the MNIST dataset, then build, compile, train, and evaluate an optimal Convolutional Neural Network (CNN) model for handwritten digit classification. Finally, visualize model predictions, training history, and summarize the overall machine learning sequence and performance.

```
import tensorflow as tf
import numpy as np

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print(f"Original training images shape: {x_train.shape}")
print(f"Original testing images shape: {x_test.shape}")
print(f"Original training labels shape: {y_train.shape}")
print(f"Original testing labels shape: {y_test.shape}")

# Normalize the pixel values
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print(f"Normalized training images shape: {x_train.shape}")
print(f"Normalized testing images shape: {x_test.shape}")

# Reshape the image data to include a channel dimension
# For grayscale images, the channel dimension is 1
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)
x_test = x_test.reshape(x_test.shape[0], 28, 28, 1)

print(f"Reshaped training images shape: {x_train.shape}")
print(f"Reshaped testing images shape: {x_test.shape}")

# Convert the labels to one-hot encoded format
num_classes = 10
```

```python
y_train = tf.keras.utils.to_categorical(y_train, num_classes)
y_test = tf.keras.utils.to_categorical(y_test, num_classes)

print(f"One-hot encoded training labels shape: {y_train.shape}")
print(f"One-hot encoded testing labels shape: {y_test.shape}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━2s 0us/step
Original training images shape: (60000, 28, 28)
Original testing images shape: (10000, 28, 28)
Original training labels shape: (60000,)
Original testing labels shape: (10000,)
Normalized training images shape: (60000, 28, 28)
Normalized testing images shape: (10000, 28, 28)
Reshaped training images shape: (60000, 28, 28, 1)
Reshaped testing images shape: (10000, 28, 28, 1)
One-hot encoded training labels shape: (60000, 10)
One-hot encoded testing labels shape: (10000, 10)
```

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Dropout

model = Sequential()

# First Conv2D layer needs input_shape
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))

# Add a Flatten layer
model.add(Flatten())

# Add Dense layers with ReLU activation
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

# Add a final Dense output layer with num_classes units and 'softmax'
activation
model.add(Dense(num_classes, activation='softmax'))

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Print the model summary
model.summary()

print("CNN model built and compiled successfully.")
```

```
/usr/local/lib/python3.12/dist-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
```

```
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

```
CNN model built and compiled successfully.
```

```
epochs = 10
batch_size = 64

# Train the model
print(f"Starting model training for {epochs} epochs with batch size
{batch_size}...")
history = model.fit(
    x_train, y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_data=(x_test, y_test)
)

print("Model training complete.")
```

```
Starting model training for 10 epochs with batch size 64...
Epoch 1/10
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━11s 7ms/step - accuracy: 0.8478 - loss: 0.4881 -
val_accuracy: 0.9819 - val_loss: 0.0567
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━3s 3ms/step - accuracy: 0.9728 - loss: 0.0927 -
val_accuracy: 0.9884 - val_loss: 0.0384
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━6s 4ms/step - accuracy: 0.9813 - loss: 0.0618 -
val_accuracy: 0.9898 - val_loss: 0.0322
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━4s 4ms/step - accuracy: 0.9852 - loss: 0.0501 -
val_accuracy: 0.9903 - val_loss: 0.0292
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━3s 3ms/step - accuracy: 0.9880 - loss: 0.0391 -
val_accuracy: 0.9905 - val_loss: 0.0261
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━3s 4ms/step - accuracy: 0.9899 - loss: 0.0337 -
val_accuracy: 0.9910 - val_loss: 0.0267
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━4s 4ms/step - accuracy: 0.9911 - loss: 0.0278 -
val_accuracy: 0.9927 - val_loss: 0.0233
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━3s 4ms/step - accuracy: 0.9909 - loss: 0.0283 -
val_accuracy: 0.9915 - val_loss: 0.0289
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━3s 4ms/step - accuracy: 0.9928 - loss: 0.0234 -
val_accuracy: 0.9916 - val_loss: 0.0286
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━4s 4ms/step - accuracy: 0.9928 - loss: 0.0211 -
val_accuracy: 0.9931 - val_loss: 0.0231
Model training complete.
```

```python
import pickle
with open('mnist_nn_model.pkl','wb') as file:
  pickle.dump(model,file)
print('model is saved')
```

```
model is saved
```

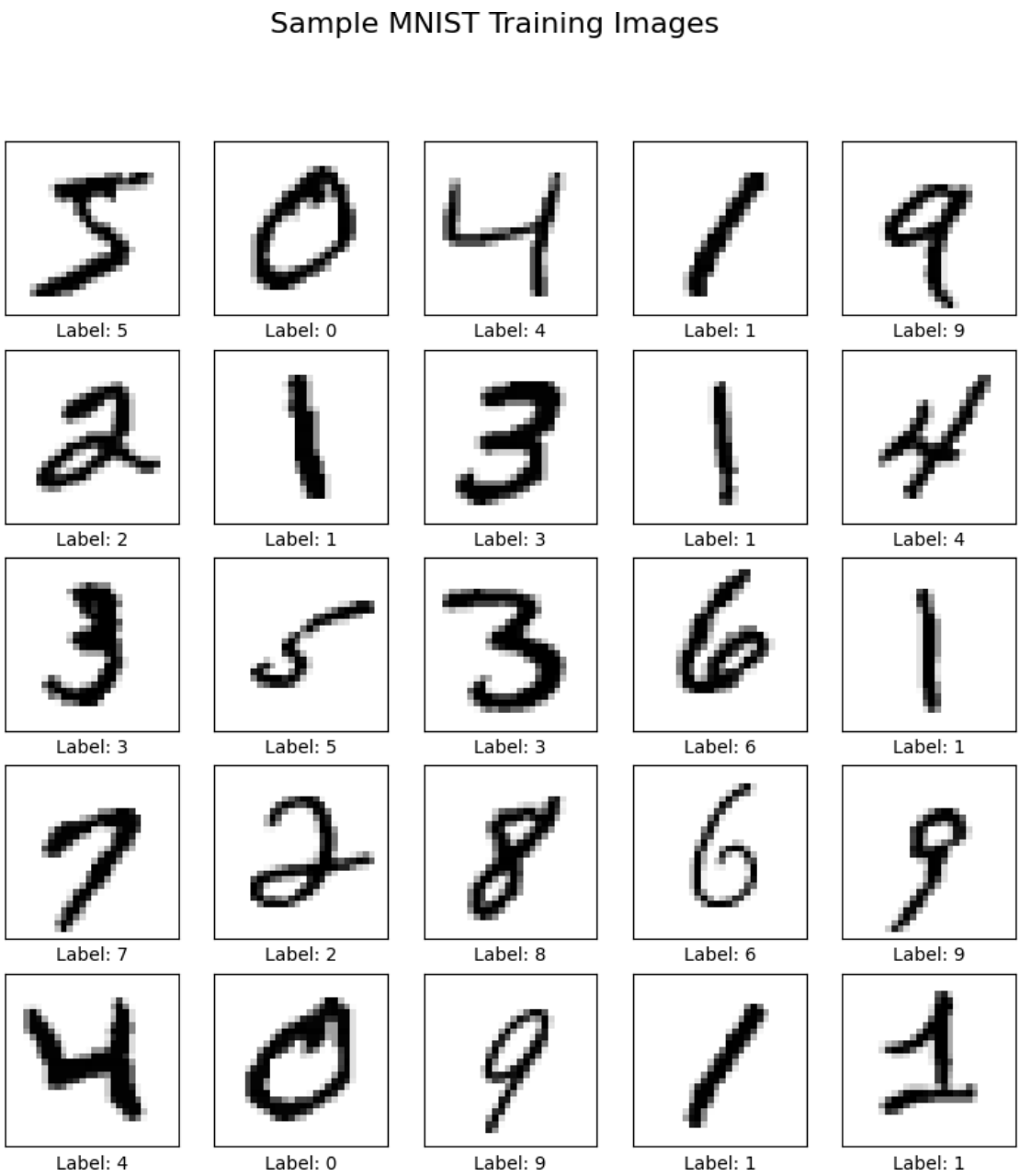
```python
import os
model_path='mnist_nn_model.pkl' # Changed filename from 'mnist_cnn_model.pkl'
to 'mnist_nn_model.pkl'
model_size_byte=os.path.getsize(model_path)
model_size_mb=model_size_byte/(1024*1024)
print(f"the size is '{model_path}' is :{model_size_mb:.2f}MB")
```

```
the size is 'mnist_nn_model.pkl' is :2.61MB
```

```python
import matplotlib.pyplot as plt
import numpy as np

# Display some sample images from the training data
```

```python
plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    # Reshape the image back to 28x28 for display
    plt.imshow(x_train[i].reshape(28, 28), cmap=plt.cm.binary)
    # Get the actual label from the one-hot encoded vector
    label = np.argmax(y_train[i])
    plt.xlabel(f"Label: {label}")
plt.suptitle("Sample MNIST Training Images", fontsize=16)
plt.show()
print("Sample MNIST dataset images plotted successfully.")
```

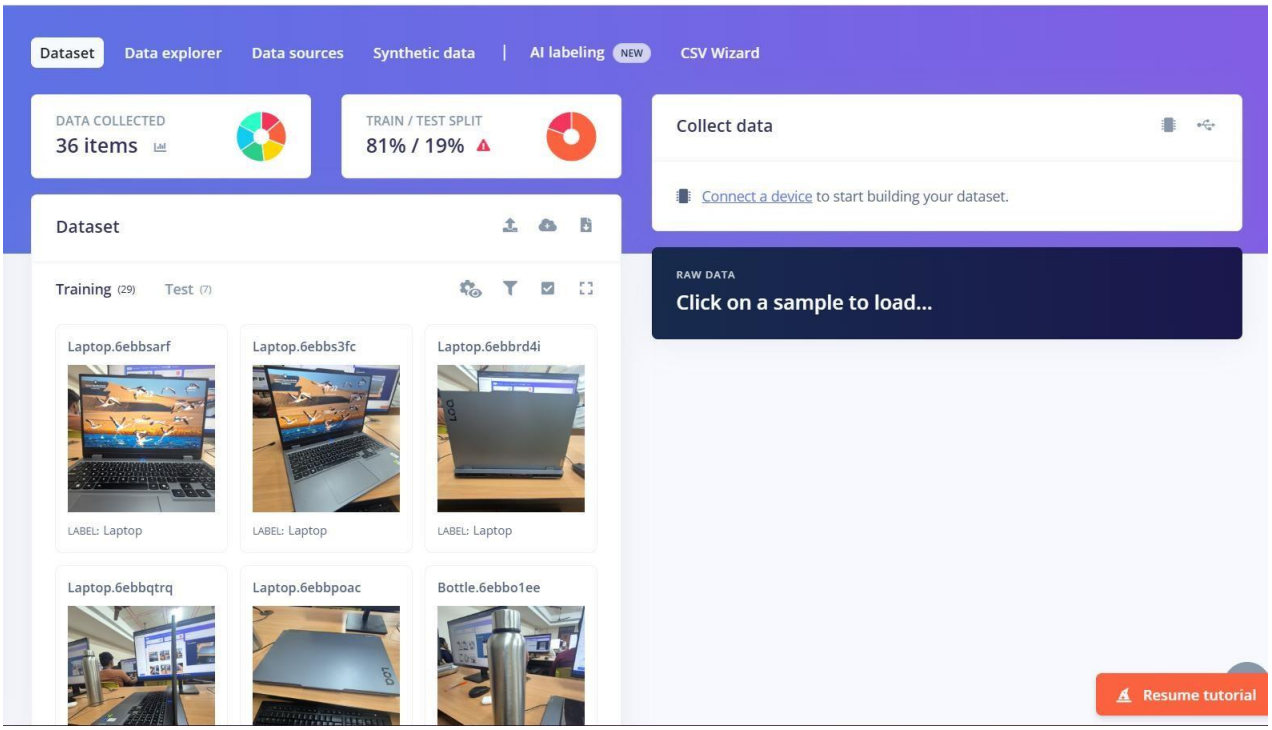



```
Sample MNIST dataset images plotted successfully.
```

The mnist_cnn_model.pkl file, which contains your trained model, has a size of **2.61 MB**.

- **Total parameters**: 225,034
- **Trainable parameters**: 225,034 (all parameters in the convolutional and dense layers are trainable)
- **Non-trainable parameters**: 0

Edge impulse task:



- Getherd 5 photos for each 6 objects and splited in train and test data on edge impluse

Edge Intelligence

Assessment 2

Name : Soni Samarth M.

Roll No : 25MML0004

Task : **Edge impulse image classification and transfer learning using edge impulse platform**

Edge impulse Data acquisition:

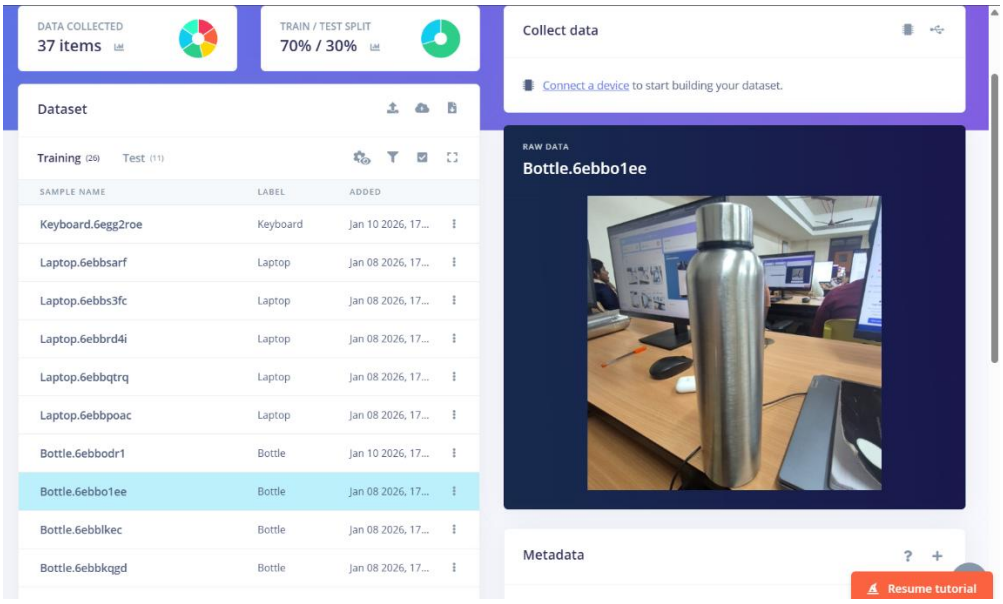
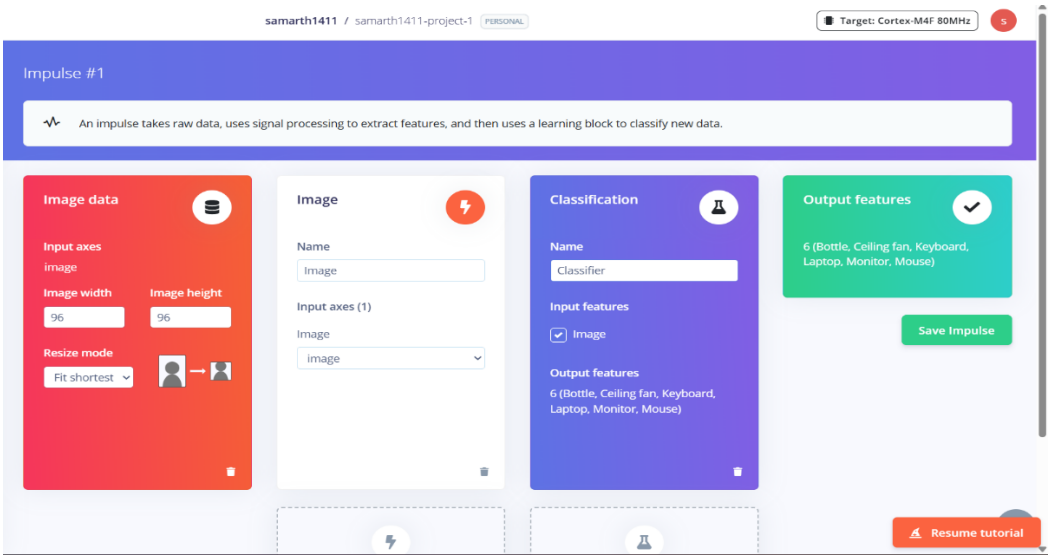
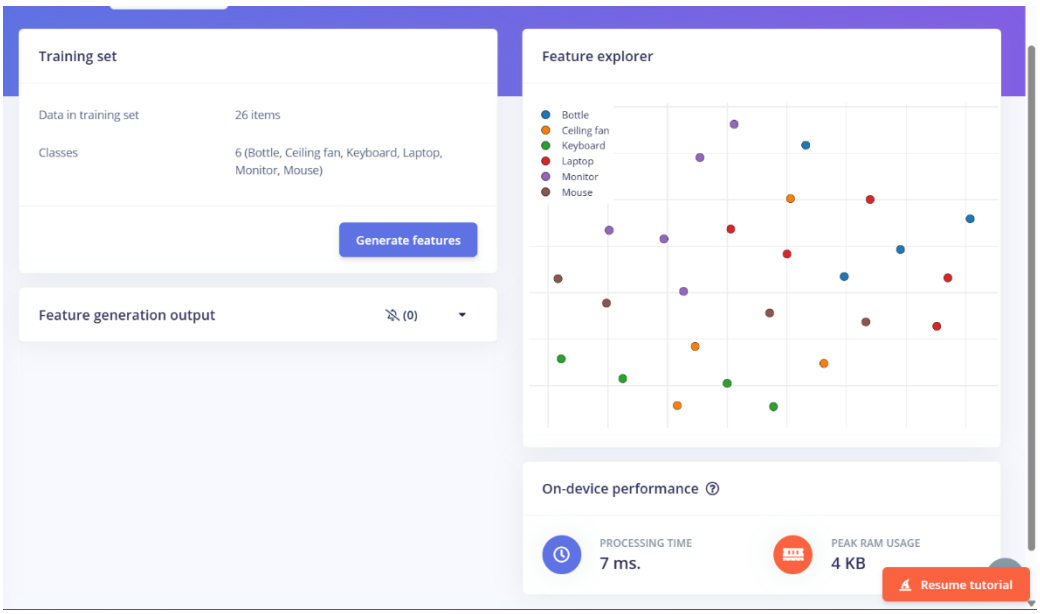


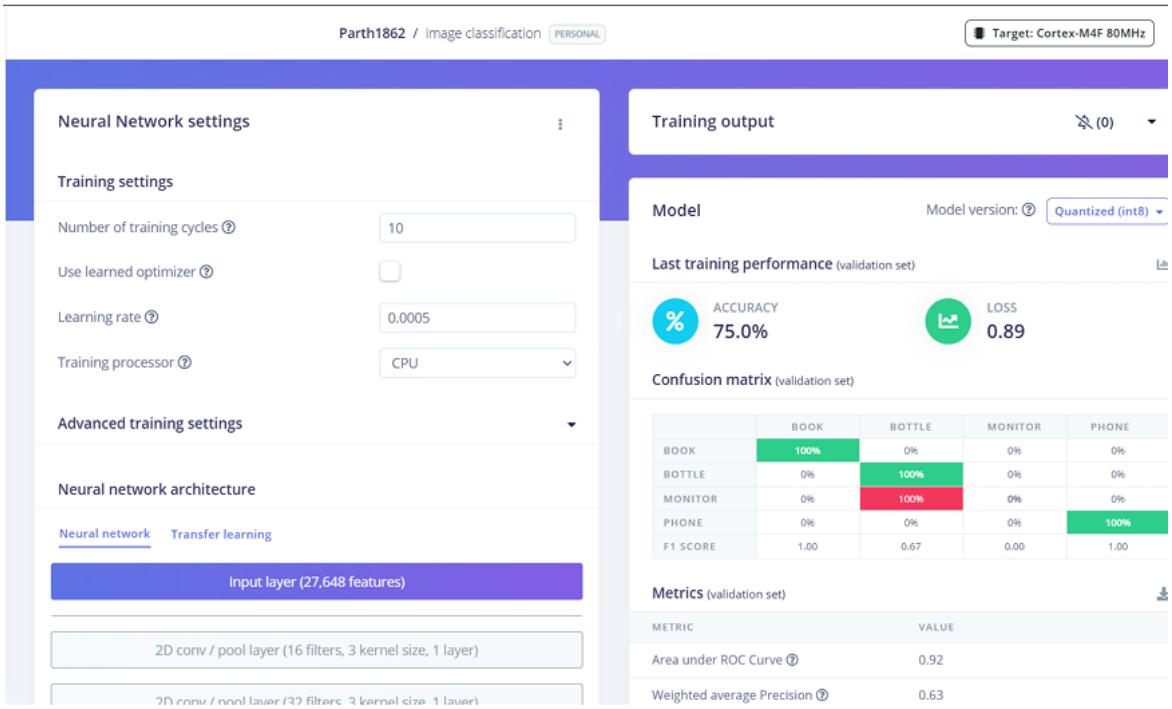
Image Classification:



- I created an image impulse where the input images are resized to 96×96 and passed into a classifier.
- The model classifies the images into 6 classes like bottle, ceiling fan, keyboard, laptop, monitor, and mouse.

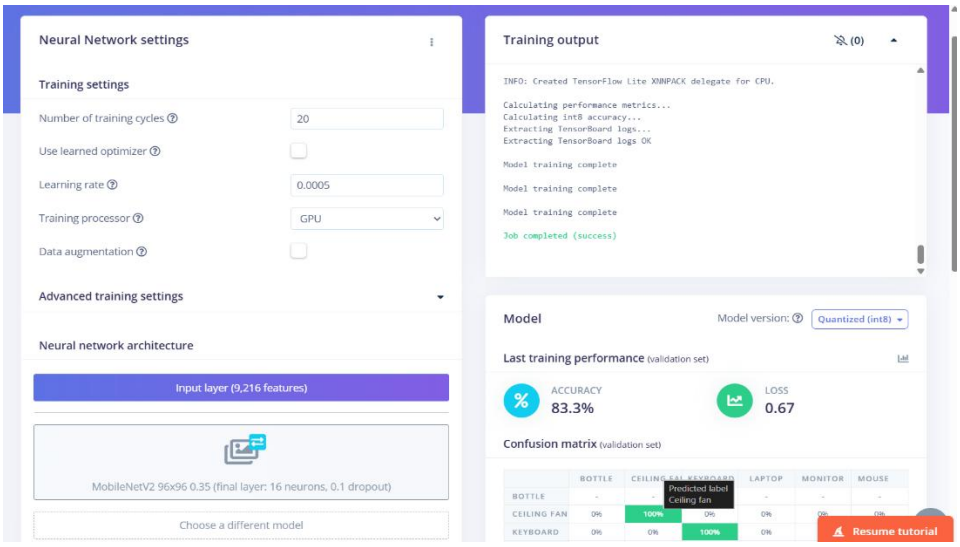
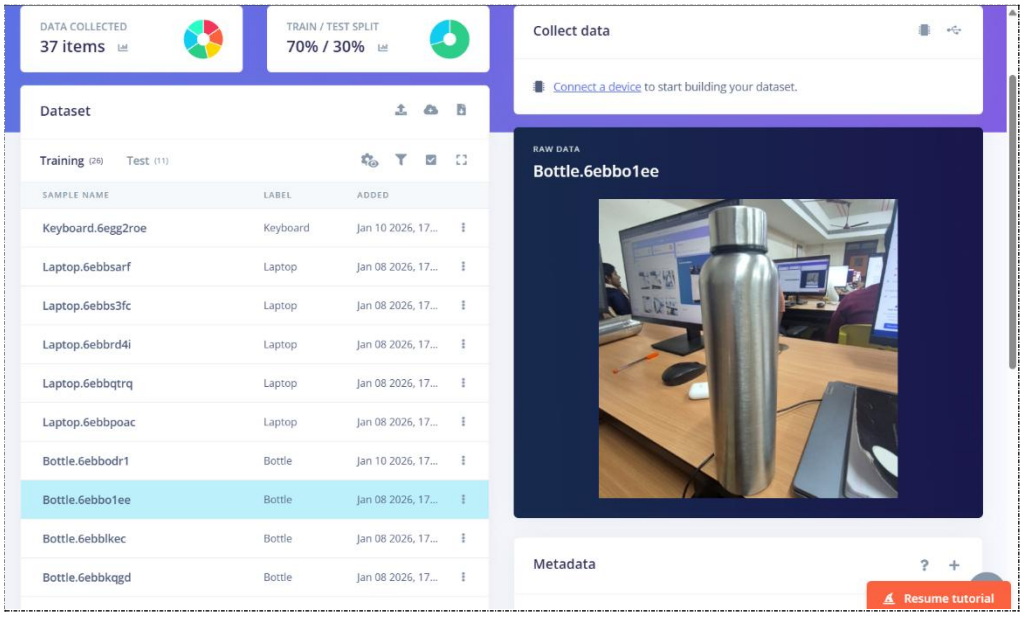


- I uploaded 26 training images across 6 classes and generated features from them. Then I visualized how each class is separated in the feature explorer and checked on-device performance like time and RAM usage.



- I trained a neural network classifier with 10 epochs using GPU and tuned the learning rate for my image dataset.
  After training, I evaluated the model using accuracy, loss, and a confusion matrix to analyze how well each class was predicted.

Transfer Learning:



- I trained the model using **transfer learning (MobileNetV2)** with 20 epochs on GPU. This improved the performance to **83.3% accuracy with lower loss**, showing better classification results.